CONLANGING WITH INKHAVEN

Developing a Constructed Language

A Beginner's Guide to Conlanging with Inkhaven

Vladimir Ulogov

2026 · examples assume Inkhaven 1.3.17 or newer

Contents

Before You Begin

This is a book about making a language — one that has never existed, with sounds you choose, words you coin, a grammar you design, and even its own alphabet. The craft is called **conlanging**, and the language you build is a **constructed language**, or **conlang**. People make conlangs for novels and games, for films, for the pure pleasure of the puzzle, or simply to understand how human language works by building one from the inside.

You will do all of it inside **Inkhaven** — a writing tool with a built-in conlang workshop. By the last page you will have taken a language from an empty project to three finished, printable books: a dictionary, a reference grammar, and a beginner's textbook for your own invented tongue.

Who this book is for

This guide assumes **no prior knowledge**. You do not need to be a linguist: every technical term is defined the first time it appears, in a marked box like the one below. You do not need to be an Inkhaven expert: we start from installing it and creating your first project. If you can type commands into a terminal and edit a text file, you have everything you need.

> TERM **Linguistics**
>
> The scientific study of human language — its sounds, its words, its grammar, and how these change over time. You will meet small, friendly pieces of it throughout this book. You do not need a background in it; that is what the term boxes are for.

How this book is organised

The book follows the natural order in which a language is built, one layer at a time. Each layer rests on the one before:

1. **Foundations** — what a conlang is, what Inkhaven is, and getting set up.
2. **The sounds** — choosing the building blocks of pronunciation and the shapes words can take.
3. **Words** — coining a vocabulary and keeping it consistent.
4. **Grammar** — how words change and combine to make meaning.
5. **History** — giving your language a past, and spawning related languages.
6. **A writing system** — designing an alphabet and compiling it into a real font.
7. **The books** — turning all of it into printed documents.
8. **A complete walkthrough** — building one small language end to end.

At the back you will find three references you will return to often: a list of every command, a list of every configuration block, and a glossary of every linguistic term used in the book.

> **HOW TO READ THE EXAMPLES**
>
> Lines you type into a terminal are shown in a monospace box and begin with the program name, like `inkhaven language init Eldar`. Configuration you write into your language is shown as `hjson` blocks. Throughout, we build one running example language called **Eldar**; in the final chapter we build a second, **Avesha**, from nothing to finished books, so you can watch the whole process at once.

What you will need

Three things, all free, all covered in Chapter 3: a copy of **Inkhaven**; a terminal to run it in; and — for the parts that use artificial intelligence, which are always optional — an **API key** for an AI provider. We will explain each when we reach it. You will also want **Typst** (a free typesetting program) if you wish to turn your finished books into PDFs, but that too can wait.

Turn the page, and let us begin with the most basic question of all: what, exactly, is a constructed language?

PART I

Foundations

CHAPTER 1

1

What is a constructed language?

A **constructed language** is a language that someone designed on purpose, rather than one that grew up naturally among a community of speakers over centuries. English, Mandarin, and Swahili are **natural languages** — nobody invented them; they evolved. Quenya (spoken by the Elves in Tolkien's books), Klingon (from **Star Trek**), and Dothraki (from **Game of Thrones**) are constructed languages — each was built, word by word and rule by rule, by a person.

TERM Constructed language (conlang)

A language deliberately created by one or more people, with an invented vocabulary and grammar, instead of arising naturally. The activity of making one is called **conlanging**, and a person who does it is a **conlanger**.

Why make one?

People build conlangs for many reasons, and yours is as good as any:

1. **Storytelling.** A believable invented language makes a fictional world feel real. A few words of a character's native tongue carry more weight than pages of description.
2. **Worldbuilding.** Place names, character names, and inscriptions all flow from a language. A consistent one makes a map or a history hang together.
3. **Art and play.** A language is an enormous, beautiful puzzle. Many people build them purely for the joy of the craft.
4. **Understanding.** Building a language teaches you, from the inside, how real languages solve the problem of turning thoughts into sound.

The parts of a language

Every human language, natural or constructed, has the same broad layers. This book has one part for each, and they build on one another in order:

TERM Phonology

The sound system of a language: which sounds it uses and how they may be combined. This is the foundation — you choose it first, because everything else is spoken (or written) using these sounds. **Part II.**

TERM **Lexicon**

The vocabulary — the full set of words. A single word in the lexicon is a **lexeme**. Building the lexicon means coining words and recording what they mean. **Part III.**

TERM **Grammar**

The rules for how words change their shape and combine into phrases and sentences. It has two halves: **morphology** (how a single word changes form, for example to mark "more than one") and **syntax** (how words are ordered into sentences). **Part IV.**

TERM **Diachronics**

The history of a language — how its sounds and words change over time, and how one language splits into a family of related daughter languages. Optional, but it is what gives a language depth and realism. **Part V.**

TERM **Writing system (orthography)**

How the language is written down: an alphabet, a syllabary, or a set of symbols, and the rules for using them. You can borrow the Latin alphabet you are reading now, or invent a script of your own. **Part VI.**

A priori and a posteriori

Conlangers use two Latin phrases you may meet elsewhere. An **a priori** language is built from scratch, owing nothing to any existing language — its words and grammar are wholly invented. An **a posteriori** language is based on real languages, like the way Esperanto draws its vocabulary from European tongues. This book builds an **a priori** language, because it is the clearest way to see every part of the process. Nothing stops you from leaning on a favourite real language for inspiration as you go.

Naturalism: should it feel real?

A common goal is a **naturalistic** conlang — one that could plausibly be a real human language, with the kinds of patterns and irregularities that natural languages have. Inkhaven is built to help here: it knows about the typical shapes of sound systems, the common ways languages mark grammar, and the natural ways sounds change over time, and it will gently steer you toward choices that feel real. But naturalism is a choice, not a rule. If you want a perfectly regular, clockwork language, you can build that too.

> **YOU DO NOT HAVE TO DO EVERYTHING**
>
> A language does not need a thousand words, a deep history, and a hand-drawn alphabet to be useful or satisfying. A few dozen well-chosen words and a simple grammar are plenty for naming places and characters in a story. Read the parts you need; the book is built so each pillar stands on its own once you have the foundations.

> **WHAT YOU LEARNED**
>
> - A **conlang** is a language built on purpose; the craft is **conlanging**.
> - Every language has layers: **phonology** (sounds), **lexicon** (words), **grammar** (rules), optional **diachronics** (history), and a **writing system**.
> - We build them in that order, each resting on the last.
> - **Naturalism** — making the language feel like a real one — is a goal you can choose or ignore.

CHAPTER 2

2

Meet Inkhaven

Inkhaven is a writing tool — a program for authors who write books, with a text editor, a place to keep notes and characters and research, and tools for turning a manuscript into a finished, formatted book. Built into it is a complete **conlang workshop**: the set of features, called the **ConLang Suite**, that this book is about. You do not need to know anything about the rest of Inkhaven to use it.

Two ways to talk to Inkhaven

Inkhaven has a full-screen interface you can run in a terminal — the **TUI**, for "text user interface" — where you write and edit. But almost everything in this book is done through **commands**: short lines you type at a terminal prompt, each beginning with the word `inkhaven`. This is deliberate. Commands are precise, they are easy to show in a book, and they are easy to repeat.

> TERM **Command line**
>
> A way of using a program by typing instructions, rather than clicking buttons. You type a line, press Enter, and the program does one thing and prints the result. The conlang tools all live under one command word: `inkhaven language …`.

Every conlang command has the same shape:

```
inkhaven language <action> <language-name> [options]
```

For example, `inkhaven language add-word Eldar makil --type noun --translation sword` runs the **add-word** action on the language named **Eldar**, with three pieces of extra information. Throughout the book, the word after `language` is the action, the next word is usually your language's name, and the parts that begin with `--` are **options** that fine-tune what happens.

> GETTING HELP ON ANY COMMAND
>
> Add `--help` to any command to see exactly what it does and what options it takes — for example `inkhaven language add-word --help`. This works for every command in the book and is the fastest way to check a detail.

Where your language lives

Inkhaven project, your work is organised into **books** — not printed books, but containers for related material. There is a special built-in one called the **Language** book, and each language you create becomes a sub-book inside it. When you create a language, Inkhaven sets up five **chapters** inside its book, each holding one kind of information:

> TERM **The Language book**
>
> Inkhaven's home for constructed languages. Creating a language with `inkhaven language init` adds a sub-book under it, pre-divided into five chapters: **Meta**, **Dictionary**, **Grammar**, **Phonology**, and **Sample texts**.

You will spend most of your time putting information into these chapters — sounds into **Phonology**, words into **Dictionary**, grammar rules into **Grammar**. The book is always the master copy of your language; the tools read from it and write back to it. We will see exactly how in the next chapter.

The role of artificial intelligence

Some of Inkhaven's conlang tools can call on an **AI** (a large language model, the kind of program behind chat assistants) to help — to invent a batch of words on a theme, to draft a letter-shape from a description, or to write a study guide. These features are powerful, but they are always **optional** and always **advisory**: the AI proposes, and nothing is saved to your language until you approve it. Wherever a command uses AI, the book says so plainly, and you can skip it.

> TERM **AI provider and API key**
>
> An **AI provider** is a company that runs a language model you can connect to (examples include DeepSeek, OpenAI, and others). An **API key** is a secret password that lets your copy of Inkhaven use that provider. You set one up once; Chapter 3 shows how. If you choose not to use AI at all, every essential part of building a language still works without it.

WHAT NEEDS AI, AND WHAT DOES NOT

Does not need AI: choosing sounds, generating word-shapes, building the lexicon by hand, all grammar and morphology, sound change and family trees, importing and compiling fonts, and producing the dictionary and grammar reference. **Uses AI (optional):** generating themed vocabulary, drafting glyph artwork from a description, reconstructing a proto-form, the grammar **study guide**, and the learner's **tutorial**.

WHAT YOU LEARNED

- Inkhaven is a writing tool with a built-in **ConLang Suite**.
- You drive the conlang tools with **commands** of the form `inkhaven language <action> <name> [options]`; `--help` explains any of them.
- Each language lives in a **Language** sub-book with five chapters: Meta, Dictionary, Grammar, Phonology, Sample texts.
- **AI** features are optional and advisory — nothing is saved without your approval, and everything essential works without AI.

CHAPTER 3

3

Getting set up

This chapter takes you from nothing to an empty language, ready to fill in. Five short steps: install Inkhaven, create a project, create your language, learn how to edit its chapters, and (optionally) connect an AI provider.

1. Install Inkhaven

Inkhaven is a single program. If you have the Rust toolchain installed, the simplest route is:

```
cargo install inkhaven
```

This downloads and builds the latest release and puts an `inkhaven` command on your system. Pre-built downloads are also available from the project's releases page. To check it worked, run:

```
inkhaven --version
```

You should see a version number of `1.3.17` or newer (this book's examples assume that release). If the command is not found, make sure the install location is on your system's `PATH`; the install step prints where it placed the program.

2. Create a project

Everything you do lives inside a **project** — a folder Inkhaven manages. Create one with `init`, giving it a path:

```
inkhaven init ~/eldar-project
```

This makes the folder, sets up Inkhaven's internal storage, and prints a short summary. From now on, run conlang commands from inside that folder:

```
cd ~/eldar-project
```

> **ONE PROJECT, MANY LANGUAGES**
>
> A single project can hold as many languages as you like — useful when you build a family of related tongues later (Part V). Each is a separate language under the same project.

3. Create your language

Now create the language itself. We will call ours **Eldar**:

```
inkhaven language init Eldar
```

This adds an **Eldar** sub-book under the Language book and creates its five chapters: **Meta**, **Dictionary**, **Grammar**, **Phonology**, and **Sample texts**. The language exists now, but it is empty — no sounds, no words, no rules. Filling those in is the rest of the book.

You can list your languages at any time:

```
inkhaven language list
```

4. How you edit a language

Here is the one mechanic that everything else depends on, so read it carefully.

Most of what defines a language — its sounds, its grammar rules, its history — is written as small blocks of structured text in a format called **HJSON**, and placed into the right chapter of the language book. HJSON is a gentle, human-friendly way to write structured data: things in `{ … }` are collections of named fields, and things in `[ … ]` are lists.

> TERM **HJSON**
>
> A relaxed, human-readable data format (a friendlier cousin of JSON). You write fields as `name: value`, group them with curly braces `{ }`, and make lists with square brackets `[ ]`. Inkhaven reads these blocks to reconstruct your language.

On disk, each chapter of your language is a folder of small text files. To add a phonology block, for example, you create a text file in the Phonology chapter's folder and paste the block in. The chapters live under your project at:

```
books/language/<your-language>/04-phonology/
books/language/<your-language>/03-grammar/
books/language/<your-language>/05-sample-texts/
```

(The exact numbers may differ; the names are what matter.) After you add or change a file by hand, tell Inkhaven to notice it:

```
inkhaven reindex --adopt
```

This **adopts** any new files into the language so the tools can read them. You run it once after editing files; the commands that write changes for you (like `add-word`) do not need it.

> **THE ONE HJSON PITFALL TO REMEMBER**
>
> In HJSON, a value that is not in quotes runs to the end of the line. So always put quotes around short word-like values such as `kind: "consonant"` or `position: "suffix"`. If you forget, you will get a clear error pointing at the line. When in doubt, quote it.

We will write our first real block — the sound inventory — in the very next chapter, so this will quickly become familiar.

5. Connect an AI provider (optional)

Skip this section entirely if you do not want to use the AI features; everything essential works without it. To enable them, you tell Inkhaven which provider to use and give it your API key. You obtain a key by signing up with a provider (this book's live examples use **DeepSeek**); the provider gives you a long secret string. You make it available to Inkhaven through an **environment variable** — a named value your terminal holds. For DeepSeek, for instance:

```
export DEEPSEEK_API_KEY="your-secret-key-here"
```

Then, on any command that uses AI, you select the provider with `--provider deepseek`. Other providers work the same way with their own key names. That is all the setup AI needs; we will use it sparingly and always say when.

WHAT YOU LEARNED

- Install with `cargo install inkhaven`; check with `inkhaven --version`.
- Create a project with `inkhaven init <path>`, then `cd` into it.
- Create a language with `inkhaven language init <name>` — it gets five chapters.
- Define a language by placing **HJSON** blocks into chapter folders, then running `inkhaven reindex --adopt`. Always quote short values.
- AI features are optional: set an API key and pass `--provider` when you want them.

PART II

The Sounds of Your Language

CHAPTER 4

4

Phonemes: the sounds

A language begins with its sounds. Before a single word can exist, you must decide what your language is **made of** — the small set of distinct sounds that its words are built from. These are its **phonemes**.

TERM Phoneme

A single distinct sound that can change the meaning of a word in a given language. In English, /p/ and /b/ are separate phonemes, because **pat** and **bat** mean different things. The slashes are the usual way to write a phoneme, to distinguish it from a letter. A language typically has somewhere between a dozen and a few dozen phonemes.

Consonants and vowels

Phonemes come in two broad families. **Vowels** are the open, sung sounds your voice glides through — the /a/ in **father**, the /i/ in **machine**. **Consonants** are the sounds made by closing or narrowing the mouth somewhere — /p/, /t/, /k/, /s/, /m/. Every word is a weave of the two.

TERM Vowel and consonant

A **vowel** is a sound made with open, unobstructed airflow (a, e, i, o, u and their many cousins). A **consonant** is a sound made by obstructing the airflow with the lips, tongue, or throat (p, t, k, s, m, n, l, r, and so on). Most syllables are one or more consonants around a vowel.

A note on the IPA

You will see phonemes written with the letters you know, but also with a few unusual symbols like /ʃ/ (the **sh** sound) or /ŋ/ (the **ng** at the end of **sing**). These come from the **International Phonetic Alphabet**, a worldwide system where one symbol always means exactly one sound. You do not need to learn it; for an a-priori language you may simply use ordinary letters for your sounds. But the IPA is handy when a sound has no obvious letter, and Inkhaven happily accepts either.

TERM **International Phonetic Alphabet (IPA)**

A standard set of symbols in which each symbol represents one and only one speech sound, used by linguists worldwide. Useful for sounds that ordinary spelling writes ambiguously (English **th** is two different sounds; IPA writes them /θ/ and /ð/). Inkhaven lets you give each phoneme an IPA symbol and a plain-letter spelling.

Building your inventory

Your full set of phonemes is your **inventory**. You declare it as a `phonemes` list inside a phonology block, placed in the **Phonology** chapter (Chapter 3 explained how to add a block). Each phoneme gives three things: its `ipa` symbol (the sound), an optional `romanize` spelling (how you will write it with ordinary letters), and its `kind` — `"consonant"` or `"vowel"`.

Here is a small starter inventory for Eldar — eight consonants and three vowels:

```
{
  phonemes: [
    { ipa: "p", romanize: "p", kind: "consonant" }
    { ipa: "t", romanize: "t", kind: "consonant" }
    { ipa: "k", romanize: "k", kind: "consonant" }
    { ipa: "s", romanize: "s", kind: "consonant" }
    { ipa: "m", romanize: "m", kind: "consonant" }
    { ipa: "n", romanize: "n", kind: "consonant" }
    { ipa: "l", romanize: "l", kind: "consonant" }
    { ipa: "r", romanize: "r", kind: "consonant" }
    { ipa: "a", romanize: "a", kind: "vowel" }
    { ipa: "i", romanize: "i", kind: "vowel" }
    { ipa: "u", romanize: "u", kind: "vowel" }
  ]
}
```

Save this as a file in your language's `04-phonology` folder, then run `inkhaven reindex --adopt`. Eldar now has a sound system.

HOW BIG SHOULD THE INVENTORY BE?

Real languages range from about eleven phonemes to over a hundred, but most sit between twenty and forty. A small, balanced set like the eleven above is an excellent place to start — it is easy to pronounce, easy to spell, and gives plenty of room for words. You can always add more later. A rough rule of thumb for a natural feel: more consonants than vowels, and at least three vowels.

Grouping sounds into classes

Many rules you will write later apply not to one sound but to a whole **family** of sounds — "all consonants", "all vowels", "the stop consonants". To name such a family, you declare a **class**: a label and the list of phonemes in it. The two most useful are a class of all consonants and a class of all vowels, usually called `C` and `V`:

```
classes: {
  C: ["p", "t", "k", "s", "m", "n", "l", "r"]
  V: ["a", "i", "u"]
}
```

Add this `classes` field inside the same phonology block, beside `phonemes`. We will use `C` and `V` in the next chapter to describe the shapes words can take.

TERM **Phoneme class**

A named group of phonemes that behave alike for some rule — for example all consonants, all vowels, or all **front** vowels. Classes let you write a rule once for a whole family instead of repeating it for each sound.

Checking your work

You can confirm Inkhaven has read your inventory with the **stats** command, which prints a profile of the language:

```
inkhaven language stats Eldar
```

Early on it will simply report your inventory size — eight consonants and three vowels. As you add words later, the same command grows into a rich picture of how your sounds are actually used.

WHAT YOU LEARNED

- A **phoneme** is a meaning-distinguishing sound; the full set is your **inventory**.
- Phonemes are **consonants** or **vowels**; you may write them with ordinary letters or **IPA** symbols.
- Declare the inventory as a `phonemes` list in the Phonology chapter; give each an `ipa`, a `romanize` spelling, and a `kind`.
- Group sounds into **classes** (like `C` and `V`) to write rules over whole families.
- `inkhaven language stats` confirms what Inkhaven read.

CHAPTER 5

5

Syllables and word shapes

You have sounds. Now you decide how they fit together into **syllables**, and from those, into possible words. This is where a language starts to have a characteristic feel — the difference between a flowing tongue of open syllables (like `ta-ki-na`) and a crunchy one full of consonant clusters (like `strkth`).

What a syllable is

A **syllable** is one beat of speech, built around a vowel. The word **banana** has three: **ba-na-na**. Each syllable has up to three parts.

> TERM **Syllable**
>
> A unit of pronunciation containing one vowel sound, optionally wrapped in consonants. It has three parts: the **onset** (consonants before the vowel), the **nucleus** (the vowel itself, the core), and the **coda** (consonants after the vowel). In **cat**, the onset is /k/, the nucleus is /a/, and the coda is /t/.

> TERM **Onset, nucleus, coda**
>
> The three slots in a syllable. The **nucleus** is the required centre (a vowel). The **onset** is the consonant(s) before it, and the **coda** the consonant(s) after it; both are usually optional. A syllable with no coda (like **ta**) is called **open**; one with a coda (like **tan**) is **closed**.

Describing word shapes with templates

How do you tell Inkhaven what a possible Eldar word looks like? With a **template**: a small pattern written with your class names, where `C` means "a consonant", `V` means "a vowel", and parentheses mean "optional". The pattern `C V (C)` reads: a consonant, then a vowel, then optionally another consonant — that is, the syllables **ta** or **tan**, but not **atra**.

> TERM **Syllable template**
>
> A pattern describing the allowed shape of a syllable, written with phoneme class names. `C V` is consonant-plus-vowel; `C V C` adds a final consonant; `(C)` marks an optional slot. A language usually allows a handful of templates.

You declare templates in the phonology block, under a `templates` field. Each template can carry a `weight` — a number saying how common that shape should be when Inkhaven generates words (higher means more frequent). Here we let Eldar have open `CV` syllables and closed `CVC` ones, with open ones twice as common:

```
templates: {
  root: [
    { pattern: "C V",   weight: 2.0 }
    { pattern: "C V C", weight: 1.0 }
  ]
}
```

The name `root` groups these as the shapes used for word **roots** (the core of a word). Add this field beside `phonemes` and `classes`, and reindex.

Generating words

With an inventory and templates, Inkhaven can invent word-shapes for you that obey your rules. This is the fastest way to discover the **sound** of your language and to find candidate words to give meanings to later.

```
inkhaven language generate-word Eldar --role root --count 10
```

This prints ten random roots that fit your templates — perhaps **taki**, **mun**, **sala**, **kira**. None of them mean anything yet; they are raw material. When one sounds right, you will turn it into a real word in Part III.

> **LISTEN TO WHAT COMES OUT**
>
> Generating a batch of words is the best way to **audition** your phonology. If the results feel too harsh, too samey, or hard to say, adjust your inventory or template weights and generate again. This loop — tweak, generate, listen — is the heart of designing a sound system. It costs nothing and uses no AI.

Inspecting a single word

To see how Inkhaven breaks a word into syllables, use the **syllabify** inspector:

```
inkhaven language syllabify Eldar --word takina
```

It prints the syllable division — `ta.ki.na` — showing the onsets, nuclei, and codas it found. This is useful for checking that your templates do what you expect, and it is the same machinery that later figures out where stress falls.

TERM **Word root**

The core form of a word, carrying its basic meaning, before any endings are added. **build** is a root; **builder** and **rebuilding** are formed from it. In Part IV you will add endings to roots; here you are just shaping the roots themselves.

WHAT YOU LEARNED

- A **syllable** is a beat of speech: an optional **onset**, a vowel **nucleus**, and an optional **coda**.
- **Templates** like `C V (C)` describe allowed syllable shapes; `weight` controls how often each appears.
- `generate-word` invents word-shapes that obey your rules — raw material for real words.
- `syllabify --word` shows how a word divides, for checking your templates.
- Designing phonology is a loop: tweak, generate, listen, repeat.

CHAPTER 6

6

Phonotactics: the rules of combination

Templates say what shape a syllable has. **Phonotactics** says which actual combinations of sounds are allowed inside that shape. Every language forbids certain sequences: English allows **spr-** at the start of a word (**spring**) but never **tlp-**; Japanese allows almost no consonant clusters at all. These rules are a big part of why languages sound the way they do.

TERM Phonotactics

The rules governing which sequences of sounds are permitted in a language — which consonants may cluster, which sounds may end a syllable, and so on. They are constraints layered on top of your syllable templates.

Constraints

You express these rules as a list of **constraints** in the phonology block, under a `constraints` field. Each constraint has a `kind`, and some take extra details. The most common kinds:

TERM Constraint

A single phonotactic rule that rejects certain words. Constraints are checked whenever Inkhaven generates or validates a word; a word that breaks any constraint is not a legal word of the language.

Limit cluster length

A **cluster** is two or more consonants in a row. To cap how long clusters may be, use `max_cluster_size`:

```
{ kind: "max_cluster_size", value: 2 }
```

This forbids three-consonant pileups while still allowing pairs like **tr** or **sk**. Setting the value to `1` forbids all clusters, giving a smooth, open-syllabled language.

Forbid doubled consonants

A **geminate** is the same consonant written twice, like the **tt** in Italian **gatto**. If you do not want them:

```
{ kind: "no_geminate" }
```

TERM **Geminate**

A doubled or lengthened consonant, as in Italian **notte** (“night”). Some languages use them meaningfully; many forbid them. `no_geminate` rules them out.

Restrict what may end a syllable

Often a language allows many consonants at the start of a syllable but only a few at the end. Use `forbid_in_coda` (or its mirror `forbid_in_onset`) with a class name. This is **syllable-aware** — Inkhaven works out the coda for you:

```
{ kind: "forbid_in_coda", classes: ["Stop"] }
```

(This assumes you have declared a class named `Stop`; you can name a class for any group of sounds you like, just as you named `C` and `V`.)

Follow the sonority rule

Real consonant clusters are not random: they tend to rise in **sonority** (roughly, loudness or openness) toward the vowel and fall away after it. That is why **pla-** feels natural and **lpa-** does not. Turn on this natural tendency with:

```
{ kind: "sonority_sequencing" }
```

TERM **Sonority**

How open and resonant a sound is. Vowels are most sonorous; then glides (w, y), liquids (l, r), nasals (m, n), and finally the quiet stops (p, t, k). The **sonority sequencing principle** says syllables tend to rise in sonority up to the vowel and fall afterward — a strong ingredient of a natural sound.

Putting it together

A complete phonotactics section for Eldar might read:

```
constraints: [
  { kind: "max_cluster_size", value: 2 }
  { kind: "no_geminate" }
  { kind: "sonority_sequencing" }
]
```

Add it to the phonology block, reindex, and generate a fresh batch of words with `generate-word`. You will notice the output is now cleaner — no triple clusters, no doubled letters, and clusters that "lean" the natural way.

CONSTRAINTS ARE A SOUND DESIGNER'S DIALS

There is no single correct set. Tight constraints (short clusters, restricted codas) give a smooth, melodic language; loose ones give a rugged, clattering one. Generate words after each change and let your ear decide. The constraints are also the gatekeeper later: when you coin or import words, anything that breaks them is flagged automatically (Chapter 10).

WHAT YOU LEARNED

- **Phonotactics** are the rules for which sound sequences are allowed.
- You write them as a list of `constraints`, each with a `kind`.
- Useful kinds: `max_cluster_size`, `no_geminate`, `forbid_in_coda` / `forbid_in_onset`, and `sonority_sequencing`.
- Tighter constraints sound smoother; looser ones sound rougher — tune by ear using `generate-word`.

CHAPTER 7

7

Allophony: sounds that shift

Here is a subtle, lovely fact about real languages: a single phoneme is often pronounced differently depending on its neighbours, without speakers even noticing. The English /t/ in **top** is a sharp, puffed sound; the /t/ in **stop** is softer; the /t/ in **butter** (in American speech) is a quick tap. Same phoneme, three pronunciations. These context-dependent variants are **allophones**, and the phenomenon is **allophony**. Adding a little of it is one of the easiest ways to make a conlang feel alive.

TERM Allophone and allophony

An **allophone** is one of the several ways a single phoneme is actually pronounced, chosen automatically by its surroundings. **Allophony** is the system of such variations. Speakers treat the allophones as "the same sound", but the surface pronunciation differs. Example: a /k/ that becomes a **ch**-sound before /i/.

Underlying and surface forms

To talk about allophony we need two ideas. The **underlying form** of a word is how it is built from phonemes — its blueprint. The **surface form** is how it is actually pronounced after the allophony rules have applied. A rule turns one into the other.

TERM Underlying form and surface form

The **underlying form** is the abstract phoneme sequence a word is made of. The **surface form** is the real pronunciation after allophony rules run. If /k/ becomes **ch** before /i/, then the word with underlying /kira/ has the surface form **chira** — but it is still "the /k/ word" underneath.

Writing a rule

Allophony rules use a compact notation borrowed from linguistics, sometimes called **SPE notation**. A rule has four parts:

```
WHAT  >  BECOMES  /  LEFT _ RIGHT
```

Read it as: "**WHAT** becomes **BECOMES** when it sits between **LEFT** and **RIGHT**". The underscore `_` marks the spot where the changing sound sits. So:

```
k > tʃ / _ i
```

means "/k/ becomes /tʃ/ (the **ch** sound) when an /i/ follows it" — the left side of the context is empty, the right side is /i/. A few more building blocks:

1. A `#` stands for a **word boundary** — the start or end of a word. `p > f / _ #` means "/p/ becomes /f/ at the end of a word".
2. An empty side means "no condition there". `_ i` cares only about what follows.
3. A context token may be a **class name** (like `V` for any vowel) rather than a single sound: `k > h / V _ V` means "/k/ becomes /h/ between two vowels".

TERM **SPE notation**

A standard shorthand for sound rules, named after a famous 1968 linguistics book (**The Sound Pattern of English**). The form is `target > result / left _ right`, where `_` is the target's position, `#` is a word edge, and a class name matches any sound in that class. Inkhaven uses it for both allophony and, later, historical sound change.

Adding allophony to Eldar

You declare rules under an `allophony` field in the phonology block. Each rule has an optional `name` (for your own reference) and the `rule` itself:

```
allophony: [
  { name: "palatalization", rule: "k > tʃ / _ i" }
  { name: "lenition",       rule: "t > s / _ i" }
]
```

Add these, reindex, and check the effect with the **ipa** inspector, which shows a word's **surface** form:

```
inkhaven language ipa Eldar --word kira
```

The underlying **kira** comes back pronounced **tʃira** — the /k/ has palatalized before the /i/. From now on, every word Inkhaven generates or inflects will have these rules applied automatically, so the variation is consistent everywhere.

ALLOPHONY RULES APPLY IN ORDER

The rules run top to bottom, and an earlier rule can feed a later one. Order them thoughtfully, and use the `ipa` inspector to confirm a tricky word comes out the way you intend. A little allophony goes a long way — two or three well-chosen rules are plenty for a natural feel.

WHY THIS MATTERS LATER

Allophony is not just decoration. The same rules fire when you add grammatical endings to words (Part IV) — so a suffix can trigger a sound change at the join, exactly as in real languages. And the **historical** sound changes of Part V use this very same notation. Learn it once; use it three times.

WHAT YOU LEARNED

- An **allophone** is a context-dependent pronunciation of a phoneme; the system is **allophony**.
- The **underlying form** is the blueprint; the **surface form** is the real pronunciation after rules apply.
- Rules use **SPE notation**: `target > result / left _ right`, with `_` the target, `#` a word edge, and class names matching families.
- Declare them under `allophony`; check results with `ipa --word`.
- The same notation reappears for grammar boundaries and for historical change.

CHAPTER 8

8

Stress, spelling, and tone

Three finishing touches complete your sound system: where the emphasis falls in a word (**stress**), how you write the language with ordinary letters (**romanization**), and — for some languages — meaningful pitch (**tone**). The first two you will almost certainly want; the third is optional.

Stress

In most languages, one syllable of a word is said a little louder or longer than the others. That is **stress**. English stress is unpredictable (**REcord** the noun versus **reCORD** the verb), but many languages place it by a simple rule — always the first syllable, or always the next-to-last.

> TERM **Stress**
>
> The relative emphasis given to one syllable of a word, by loudness, length, or pitch. A **fixed** stress rule places it predictably; common rules are **initial** (first syllable), **final** (last), **penultimate** (second-to-last), and **antepenultimate** (third-to-last).

You set a stress rule with the `stress` field in the phonology block. The value is one of `initial`, `final`, `penultimate`, `antepenultimate`, or `latin` (a weight-sensitive rule used in Latin). Eldar will stress the second-to-last syllable:

```
stress: "penultimate"
```

Check it with the **stress** inspector, which marks the stressed syllable:

```
inkhaven language stress Eldar --word takina
```

It returns something like `ta.'ki.na` — the mark `'` sits before the stressed syllable.

Romanization: writing it down

Your phonemes may include symbols like /ʃ/ that you do not want to type constantly. **Romanization** is the system for writing the language with ordinary Latin letters — the **romanize** spellings you already gave each phoneme are the simple version of this. For anything trickier, you can define a named **romanization scheme**.

TERM **Romanization**

A way of writing a language's sounds using the Latin alphabet (the letters a–z). The simplest romanization gives each phoneme a spelling; a fuller scheme can spell one sound several ways depending on context, the way Italian writes /k/ as **c** before **a** but **ch** before **e**.

A scheme maps phonemes to letters, and can include **contextual** rules — a spelling that depends on the surrounding sounds. Suppose you want /k/ and /s/ both written `c`, with `c` read as /s/ before a front vowel (as in Italian or English **city**):

```
romanizations: [
  { name: "default"
    mappings: [ { ipa: "k", roman: "c" }, { ipa: "s", roman:
"c" } ]
    contextual: [ { roman: "c", ipa: "s", before: "FrontV" } ] }
]
default_romanization: "default"
```

You can then convert text either way with the **romanize** inspector:

```
inkhaven language romanize Eldar --text cace
inkhaven language romanize Eldar --text /kase/ --reverse
```

The first reads spelled text into sounds; the second (`--reverse`) writes sounds back out as spelling. For most languages the simple per-phoneme `romanize` fields are all you need, and you can skip schemes entirely.

Tone (optional)

Some languages — Mandarin, Yoruba, Thai — use **pitch** to distinguish words: the same syllable said with a rising versus a falling pitch means different things. That is **tone**. If your language has no tone, skip this section; most conlangs do.

TERM Tone

The use of pitch (the rise and fall of the voice) to distinguish word meanings. In a **tonal** language, **ma** on a high pitch and **ma** on a falling pitch are different words. Tones can also interact: a **tone sandhi** rule changes one tone next to another.

To add tone, declare a `tone` block with the tones you use and any **sandhi** rules (written in the same SPE notation as allophony):

```
tone: {
  kind: "contour"
  tones: ["1", "2", "3", "4"]
  sandhi: [ { rule: "3 > 2 / _ 3" } ]
}
```

This gives four tones (numbered) and one sandhi rule — "a third tone becomes a second tone before another third tone", the famous Mandarin pattern. Try it with:

```
inkhaven language tone Eldar --tones "3 3 3"
```

YOUR PHONOLOGY IS DONE

With sounds, syllable shapes, phonotactics, allophony, stress, and (optionally) romanization and tone, your sound system is complete. Everything from here on — words, grammar, history, writing — is spoken with these sounds and obeys these rules automatically. This is the foundation the whole language stands on.

WHAT YOU LEARNED

- **Stress** is the emphasised syllable; set a fixed rule with the `stress` field (`initial`, `final`, `penultimate`, …).
- **Romanization** writes the language in Latin letters; per-phoneme `romanize` spellings suffice for most languages, with `romanizations` schemes for harder cases.
- **Tone** (optional) uses pitch to distinguish words; declare a `tone` block with tones and `sandhi` rules.
- Inspectors `stress`, `romanize`, and `tone` let you check each at a word.

PART III

Words

CHAPTER 9

9

Building a vocabulary

Your language can now make word-shapes; it is time to give them meanings. The collection of words and their meanings is the **lexicon**, and each entry pairs a word with what it means, what part of speech it is, and any extra notes you care to keep. This chapter is about adding words by hand; the next adds two more powerful ways to grow the lexicon.

Adding a word

The basic command is **add-word**. It needs the language, the word itself, its **part of speech**, and its meaning (the **translation** into your everyday language):

```
inkhaven language add-word Eldar makil --type noun --translation
sword
```

This stores **makil** in Eldar's **Dictionary** chapter as a noun meaning "sword". No reindex is needed — commands that write changes for you handle that themselves. Add a few more:

```
inkhaven language add-word Eldar kira --type noun --translation
bird
inkhaven language add-word Eldar nami --type verb --translation
see
inkhaven language add-word Eldar mira --type adjective --
translation bright
```

TERM **Part of speech**

The grammatical category of a word — what **kind** of word it is. The common ones are **noun** (a thing: sword, bird), **verb** (an action: see, run), **adjective** (a description: bright, cold), and **adverb**. The category matters later, when grammar rules apply only to words of a certain kind.

TERM **Translation (gloss)**

The meaning of a word given in your working language — for **makil**, the English "sword". A short meaning like this is also called a **gloss**. It is how you and Inkhaven know what the word means.

Recording more than the basics

A word often carries more than a bare meaning. Is it formal or vulgar? What subject does it belong to — warfare, cooking, religion? Does it belong to an older era of the language? You can record all of this, and search by it later. A dictionary entry, stored as a small HJSON block, can hold these extra fields:

```
{ word: "makil", type: "noun", translation: "sword",
  register: "formal", domain: ["weapon"], era: "third_age" }
```

TERM **Register**

The level of formality or social setting a word belongs to — **formal**, **neutral**, **vulgar**, **sacred**, **archaic**. The same idea in your everyday language separates "purchase" (formal) from "buy" (neutral). Marking register lets a language have words that fit a king's court and others that fit a tavern.

TERM **Domain**

The subject area a word belongs to — **weapon**, **kinship**, **weather**, **magic**. Domains help you build vocabulary by theme and find related words quickly.

Finding words again

As the lexicon grows you will want to search it. The **query** command filters by any of the rich fields — part of speech, register, domain, era, or a substring of the word or its meaning:

```
inkhaven language query Eldar --pos noun
inkhaven language query Eldar --domain weapon
inkhaven language query Eldar --text bright
```

Each prints the matching entries. With `--json` you get machine-readable output, handy for scripts.

Importing many words at once

If you already have a word list — perhaps in a spreadsheet — you can bring it in all at once. Export it as a CSV file (a simple table of word, part of speech, translation, …) and import:

```
inkhaven language add-word Eldar --import words.csv
```

Every row becomes an entry. This is the fast path when you are moving an existing list into Inkhaven.

> **REMOVING A WORD**
>
> Made a mistake, or retired a word? `inkhaven language remove-word Eldar makil` deletes it from the dictionary. Like `add-word`, it finds the entry by name and needs no reindex.

Using your words while you write

The reason to build a lexicon is, of course, to **use** it. When you write prose in Inkhaven's editor, you can drop an invented word straight in: type a colon, your language's name or code, and another colon — `:eldar:` — and a picker appears listing your words; choose one and it is inserted. Inkhaven also lights up your invented words where they appear in a manuscript, and can translate whole paragraphs into or out of the language. These editor features are beyond the scope of this command-focused book, but it is worth knowing the lexicon you are building is meant to be lived in.

> **WHAT YOU LEARNED**
>
> - The **lexicon** is your vocabulary; each entry pairs a word with a meaning.
> - `add-word <lang> <word> --type <pos> --translation <meaning>` adds one.
> - Record **part of speech**, **register**, **domain**, and **era**; search them with `query`.
> - Import a CSV with `add-word --import`; remove with `remove-word`.
> - In the editor, `:lang:` inserts a word from the lexicon.

CHAPTER 10

10

A healthy lexicon

A growing vocabulary can drift into trouble: two words that sound identical, two words that mean the same thing, a word that quietly breaks your own sound rules. This chapter shows how Inkhaven keeps the lexicon honest — and how it can invent whole batches of new words for you, on a theme, without ever creating a duplicate.

Auditing for problems

The **audit** command is your lexicon's health check. It reads every entry and reports three kinds of trouble:

```
inkhaven language audit Eldar
```

1. **Phonotactic violations** — a word whose sounds break the constraints you set in Chapter 6 (perhaps imported from an old list before you tightened the rules).
2. **Homophones** — two different words that come out pronounced the same after allophony. Sometimes you want these; often they are accidents.
3. **Duplicate meanings** — two words with the same translation, which may be an oversight.

> TERM **Homophone**
>
> A word that sounds the same as another but means something different — like English **to**, **too**, and **two**. A few are natural; many by accident usually signal a mistake. The audit catches them even when two words only **become** identical after your allophony rules apply.

The audit only reports; it changes nothing. Use it whenever the lexicon grows, or before you publish your dictionary, to catch surprises.

Generating words with AI

Building a few hundred words by hand is slow. Inkhaven can generate a batch for you on a theme — but with an important guarantee about **where the words come from**. The **forms** (the actual sound-shapes) are made by Inkhaven's own generator, so they always obey your phonotactics. The AI only supplies the **meanings**. This split — forms from the engine, meanings from the AI — is what keeps generated words consistent with your language.

```
inkhaven language generate-lexicon Eldar --topic seafaring --
count 20
```

This proposes twenty seafaring words, each a legal Eldar form with an AI-chosen meaning. By default it only **prints** the proposals; add `--yes` to actually add them to the dictionary.

TERM **The dedup gate**

A set of automatic checks every proposed word passes before it is accepted, so generation never creates a duplicate. It rejects a word that breaks the phonotactics, that sounds identical to an existing word, that repeats a meaning already in the lexicon, or — with the optional `--semantic` check — that is too close in meaning to an existing word (a near-synonym). "Forms obey the language; meanings come from the AI; nothing duplicates."

Catching near-synonyms

Two words can have different translations yet mean almost the same thing — "stone" and "rock", say. To reject such near-synonyms too, add `--semantic`:

```
inkhaven language generate-lexicon Eldar --topic seafaring --
count 20 --semantic --yes
```

This compares the meaning of each proposal against your existing words using a language-model technique and drops the ones that are too close. It is slower but keeps a tight, non-redundant vocabulary.

AI HERE IS ADVISORY, LIKE EVERYWHERE

Without `--yes`, generation shows you the proposals (and the reasons any were rejected) and changes nothing. You stay in control: review the list, then commit. Meanings come back in your working language. If you would rather not use AI at all, build the lexicon by hand with `add-word` — the audit and the phonotactic checks work just the same.

Finding undefined words in your writing

If you have been writing prose that **uses** the language, Inkhaven can scan your manuscript for words that look like your language but are not yet in the dictionary — names and terms you coined on the fly and never recorded:

```
inkhaven language scan-manuscript Eldar
```

It lists candidate undefined words, so you can add the ones worth keeping. This closes the loop between writing and the lexicon.

WHAT YOU LEARNED

- `audit` reports phonotactic violations, **homophones**, and duplicate meanings — it only reports, never changes.
- `generate-lexicon --topic … --count …` invents themed words: **forms** from the engine (always legal), **meanings** from the AI.
- The **dedup gate** blocks illegal, homophone, and duplicate-meaning words; `--semantic` also blocks near-synonyms.
- Nothing is added without `--yes`; AI is optional.
- `scan-manuscript` finds language-like words in your prose that are not yet defined.

PART IV

Grammar

CHAPTER 11

11

Morphology: words that change

Words are not frozen. **Cat** becomes **cats**; **walk** becomes **walked**. A word changes its shape to express grammar — number, tense, who is doing what. **Morphology** is the study of those changes, and it is the first half of grammar. This chapter shows how to build the pieces words are made of and how to assemble them.

TERM **Morphology**

The part of grammar dealing with the internal structure of words — how they are built from smaller meaningful pieces, and how they change shape to express grammar. (The other half of grammar, **syntax**, deals with word order, covered in Chapter 13.)

Morphemes and affixes

The smallest meaningful pieces of a word are **morphemes**. **Cats** has two: **cat** (the thing) and **-s** (meaning "more than one"). A morpheme like **-s** that attaches to a word is an **affix**. Affixes that go on the front are **prefixes** (**re-** in **rebuild**); on the end, **suffixes** (**-s**, **-ed**); inside, **infixes**.

TERM **Morpheme**

The smallest unit of a word that carries meaning. **Unhappiness** has three: **un-** (not), **happy**, and **-ness** (the quality of). A **root** is the central morpheme; **affixes** attach to it.

TERM **Affix**

A morpheme attached to a root to modify its meaning or grammar. A **prefix** attaches to the front (**re-do**), a **suffix** to the end (**cat-s**), an **infix** in the middle. Inkhaven supports prefixes and suffixes.

Declaring your affixes

You list a language's affixes in a **morphology block** in the **Grammar** chapter. Each **morpheme** gives an `id` (a short name you choose), a `gloss` (a label for what it means, conventionally in small capitals), a `form` (the actual sounds it adds), and a `position` (`"prefix"` or `"suffix"`):

```
{
  morphemes: [
    { id: "pl",  gloss: "PL",  form: "i",  position: "suffix" }
    { id: "dat", gloss: "DAT", form: "ti", position: "suffix" }
    { id: "def", gloss: "DEF", form: "na", position: "prefix" }
  ]
}
```

Here `pl` is a plural suffix **-i**, `dat` a dative suffix **-ti**, and `def` a definite prefix **na-**. The glosses `PL`, `DAT`, `DEF` are the standard linguistic abbreviations for "plural", "dative", and "definite".

Ordering stacked affixes

When two or more affixes pile onto the same side of a root, which comes first? In many languages the order is fixed — a case ending might always sit closer to the root than a number ending. You control this with an optional `precedence` number on each morpheme:

```
{ id: "pl",  gloss: "PL",  form: "i",  position: "suffix",
precedence: 2 }
{ id: "dat", gloss: "DAT", form: "ti", position: "suffix",
precedence: 1 }
```

TERM Affix ordering (precedence)

A number saying how close an affix sits to the root when several stack: `0` (the default) means any position — the order you listed them in is kept; `1` means immediately next to the root; `2` the next slot out; and so on. A lower non-zero number is closer to the root. Above, the case suffix (precedence 1) always hugs the root and the number suffix (precedence 2) sits outside it, no matter which order a paradigm lists them.

TERM Gloss (grammatical)

A short label for a grammatical piece, written in small capitals by convention: PL (plural), SG (singular), DAT (dative case), PST (past tense), and so on. Glosses let you write what a word **means** grammatically, piece by piece.

> **TERM Inflection**
>
> Changing a word's form to express grammar **without** making it a new word — **cat** / **cats**, **walk** / **walked**. The set of all inflected forms of a word is its **paradigm**. (Contrast with **derivation**, Chapter 12, which makes a genuinely new word.)

Paradigms: the forms of a word

A **paradigm** is the full table of a word's inflected forms — for a noun, perhaps singular and plural, in each grammatical case. You describe one as a list of **cells**, each saying which features it expresses (number, case) and which morphemes build it:

```
paradigms: [ { name: "noun", cells: [
  { features: { number: "sg", case: "nom" }, morphemes: [] }
  { features: { number: "pl", case: "dat" }, morphemes: ["dat",
"pl"] }
] } ]
```

Add this beside `morphemes` in the Grammar chapter. The first cell is the bare root (no morphemes); the second stacks the dative and plural suffixes.

> **TERM Paradigm**
>
> The organised set of all inflected forms of a word — for example a noun in singular and plural across every case. Each form is one **cell** of the paradigm, defined by its grammatical **features** and the morphemes that build it.

Generating the forms

Now Inkhaven can build the whole paradigm for any root, applying the morphemes **and** your allophony rules at the joins:

```
inkhaven language paradigm Eldar --root kata --template noun --
gloss stone
```

For the root **kata** ("stone") this prints each cell's surface form and its gloss — for example **katati** for "stone-DAT". And here is the elegant part: if your allophony rule says /t/ softens to /s/ before /i/, then **kata** plus the dative **-ti** comes out as **katasi**, because the rule fires at the boundary, exactly as it would in a real language. The sound machinery from Part II is working for you automatically.

Interlinear glossing

When you write a sentence in the language, you will often want to show, word by word, what each piece means — a **interlinear gloss**, the two-line format linguists use. A dictionary entry can declare which paradigm it inflects by (`paradigm: "noun"`), and then Inkhaven can gloss running text:

```
inkhaven language gloss Eldar --text "kata katai katat"
```

It prints each word above its Leipzig-style gloss — `kata` → `stone`, `katat` → `stone-DAT` — and it recognises inflected **and** allophony-altered forms, because it works out each entry's paradigm forward and matches what it finds.

> TERM **Interlinear gloss (Leipzig glossing)**
>
> A way of showing a sentence in an unfamiliar language with a word-by-word breakdown lined up underneath, using grammatical glosses (PL, DAT, ...). The name comes from the widely used **Leipzig Glossing Rules**. It is how grammars and textbooks make a foreign sentence transparent.

Beyond prefixes and suffixes

Not every language marks grammar by gluing pieces to the ends of words. Inkhaven supports the other common strategies too; you choose them on a morpheme with `position` or `process`.

TERM Infix

An affix inserted **inside** the root rather than at an edge. Tagalog forms an actor from **sulat** (“write”) as **s-um-ulat** — the **-um-** sits after the first consonant. Declare `position: "infix"`; the `anchor` is `before_first_vowel` (the default) or `after_first_vowel`.

TERM Circumfix

A single affix in two pieces that wrap around the root — German makes a past participle with **ge-…-t** (**ge-sag-t**). Declare `position: "circumfix"` and write the `form` with a `_` marking where the stem goes: `ge_t` ⟶ **ge** + stem

1. **t**.

TERM Ablaut

Marking grammar by **changing a sound inside** the root instead of adding anything — English **sing** / **sang** / **sung**. Declare `process: "ablaut"` and give the change as an SPE `rules` list (Chapter 7), e.g. `i > a`. The vowel swaps in place.

TERM Reduplication

Marking grammar by **repeating** part (or all) of the root — Malay **buku** “book” ⟶ **buku-buku** “books”. Declare `process: "reduplication"` with a `reduplicate` mode: `full` (the whole stem doubled), `initial_cv` (the first consonant + vowel copied to the front), `initial_syllable`, or `final_syllable`.

All four are written as ordinary morphemes and used in paradigm cells exactly like prefixes and suffixes — and allophony still applies across the new seams. For example, a morpheme `{ id: "ag", gloss: "AG", form: "um", position: "infix" }` turns the root **tanik** into **t-um-anik**.

Agreement

Often one word must echo the grammar of another: an adjective takes its noun's number and case, a verb takes its subject's person and number. This matching is called **agreement** (or **concord**), and it is what makes "these tall trees" work where "this tall tree" also does.

> TERM **Agreement (concord)**
>
> The requirement that a **dependent** word copy certain grammatical features from the **head** it modifies — an adjective agreeing with its noun, a verb with its subject. The shared features (number, case, gender, person) must match.

You declare agreement as rules in the morphology block: which part of speech agrees with which, on which features, and through which paradigm:

```
agreement: [
  { dependent: "adjective", head: "noun", features: ["number",
"case"], paradigm: "adj" }
]
```

Then Inkhaven can inflect a dependent to agree with a given head. Suppose a noun is plural; to get the matching form of the adjective **mira** ("bright"):

```
inkhaven language agree Eldar --word mira --pos adjective \
    --gloss bright --features "number=pl"
```

It finds the agreement rule for adjectives, picks the `adj` paradigm cell that matches `number=pl`, and prints the agreeing form — **mirai**, glossed `bright-PL`. The `--features` you pass are the head's; only the ones the rule lists as agreement features are copied.

WHAT YOU LEARNED

- **Morphology** is how words change shape; the pieces are **morphemes**, and attached ones are **affixes** (prefix / suffix / **infix** / **circumfix**).
- Beyond gluing affixes, a morpheme can be a **process**: **ablaut** (an internal sound change) or **reduplication** (repeating part of the root).
- **Inflection** changes a word's form; the full set is a **paradigm**, described as `cells` of features + morphemes.
- `paradigm` builds the forms (allophony applies at every seam); `gloss` shows a sentence interlinear, word by word.
- **Agreement** makes a dependent copy a head's features; `agree` inflects it to match.

CHAPTER 12

12

Building new words

Inflection (last chapter) changes a word's form without changing what it fundamentally is — **stone** and **stones** are the same word. **Derivation** is different: it makes a genuinely **new** word from an old one. **Build** becomes **builder** (a person who builds); **king** becomes **kingdom** (the realm of a king). This is one of the most productive ways a real lexicon grows, and your language can do it too.

TERM **Derivation**

Forming a new word (a new **lexeme**, with its own dictionary entry) from an existing one, usually by adding an affix and often changing its part of speech. **teach** (verb) → **teacher** (noun); **happy** (adjective) → **happiness** (noun). Contrast with **inflection**, which only changes a word's form, not its identity.

Declaring a derivation rule

You add derivation rules to the morphology block in the **Grammar** chapter, under a `derivations` field. Each rule names the affix it adds, where it goes, which part of speech it applies to (`from_pos`), what part of speech it produces (`to_pos`), and a template for the new meaning:

```
derivations: [
  { name: "agent", form: "ron", position: "suffix",
    from_pos: "verb", to_pos: "noun", gloss_template: "one who {}
s" }
]
```

This is an **agent noun** rule: take a verb, add **-ron**, and you get a noun meaning "one who does the verb". The `{}` in the template is filled with the root's meaning — so a verb meaning "build" yields a noun glossed "one who builds".

TERM **Agent noun**

A noun naming the doer of an action, derived from a verb — **teach** → **teacher**, **bake** → **baker**. The "**-er**" of English is an agent suffix. Agent nouns are one of the most common derivations across the world's languages.

Coining derived words

With a rule in place, ask Inkhaven to apply it to a root:

```
inkhaven language derive Eldar --root kata --gloss build --pos
verb
```

Inkhaven runs every derivation rule whose `from_pos` matches "verb", applies the affix (with allophony at the join), and prints the proposed new words — their form, meaning, and part of speech. For **kata** "build" it might propose **kataron**, "one who builds", a noun.

By default this only **proposes**. To actually add the derived words to your dictionary — recording where each came from, as a small etymology — add `--yes`:

```
inkhaven language derive Eldar --root kata --gloss build --pos
verb --yes
```

TERM **Etymology**

The origin and history of a word — what it was derived or descended from. When Inkhaven coins a derived word for you, it records the etymology in the entry ("derived from **kata** via the agent rule"), so your dictionary remembers how each word came to be.

DERIVATION VERSUS GENERATION

Two ways to grow vocabulary, for two purposes. **Generation** (Chapter 10) invents brand-new roots on a theme — good for filling out basic vocabulary. **Derivation** grows words **from words you already have**, the way real languages spin families of related terms from a single root. Use generation for breadth, derivation for depth and internal consistency.

WHAT YOU LEARNED

- **Derivation** makes a new word from an existing one, often changing its part of speech (**build** → **builder**).
- Declare rules under `derivations`: `form`, `position`, `from_pos`, `to_pos`, and a `gloss_template` with `{}` for the root's meaning.
- `derive --root … --gloss … --pos …` proposes derived words; `--yes` adds them with their **etymology** recorded.
- Use generation for new roots, derivation to grow word families from existing roots.

CHAPTER 13

13

Typology: the shape of your grammar

Beyond individual words lies the architecture of a language: does the verb come before or after the object? Does it mark the subject of a sentence specially? Does it have grammatical gender? These big design choices are the subject of **typology**, and making them is how you give your grammar a coherent character. Inkhaven gives you a questionnaire of the major choices, drawn from how real languages actually vary.

TERM Typology

The classification of languages by their structural features — word order, how they mark who-does-what, whether they have case or gender, and so on. **Typology** is, in effect, the menu of high-level design decisions every language makes. Linguists catalogue these in surveys like WALS, the **World Atlas of Language Structures**, which Inkhaven's questionnaire follows.

The questionnaire

Run the grammar command with no options to see the catalogue of sixteen features, your language's current answers, and how much you have filled in:

```
inkhaven language grammar Eldar
```

The features include word order, alignment, case, gender, number, definiteness, tense, aspect, mood, evidentiality, negation, question formation, and relative clauses. You do not have to answer all of them; answer the ones that matter to your language and leave the rest.

Three choices worth understanding

Three of the features shape a grammar more than any others. Let us define them, since they are the ones a newcomer most often meets.

TERM Word order

The default order of the **subject** (S, the doer), **verb** (V, the action), and **object** (O, the thing acted on) in a basic sentence. English is **SVO**: "the cat (S) sees (V) the bird (O)". Japanese and Latin are **SOV**: "the cat the bird sees". The six possible orders (SOV, SVO, VSO, …) are unevenly common across the world; SOV and SVO are by far the most frequent.

TERM **Alignment**

How a language marks the **subject** of a sentence. In a **nominative–accusative** language (like English), the doer is treated the same whether or not there is an object ("**she** runs", "**she** sees it"), and the object is marked differently. In an **ergative–absolutive** language, the lone subject of "she runs" is instead grouped with the **object** of "she sees it". This sounds exotic but is common — Basque and many others work this way. It is one of the most character-defining choices you can make.

TERM **Grammatical case**

Marking a noun's **role** in the sentence by changing its form (usually with an ending), rather than relying on word order. Latin **puella** "girl" becomes **puellam** as a direct object, **puellae** as a possessor. Common cases include **nominative** (subject), **accusative** (object), **dative** (recipient, "to/for"), and **genitive** (possessor, "of"). You built a dative suffix back in Chapter 11; declaring `case: yes` here records that your language has a case system.

Answering a feature

You set an answer with `--set feature=value`. The answer is checked against the catalogue, so you cannot set an invalid one:

```
inkhaven language grammar Eldar --set word_order=sov
inkhaven language grammar Eldar --set
alignment=nominative_accusative
inkhaven language grammar Eldar --set case=yes
```

Each answer is stored in the **Grammar** chapter. They are not just notes: Inkhaven's grammar book and study guide (Part VII) read these answers and explain them, and the AI translator uses them to put words in the right order.

LET YOUR CHOICES REINFORCE EACH OTHER

Typological features tend to cluster in real languages — SOV languages, for instance, usually put adjectives and possessors **before** their nouns and use **postpositions** (“the house behind” rather than “behind the house”). The questionnaire shows the typical consequences of each answer, nudging you toward a grammar that hangs together naturally. You are free to break the patterns — but knowing them helps you break them on purpose.

WHAT YOU LEARNED

- **Typology** is the set of high-level structural choices a grammar makes; `grammar` lists sixteen, WALS-aligned.
- **Word order** (SOV, SVO, …) sets the default subject–verb–object sequence.
- **Alignment** (nominative–accusative vs ergative–absolutive) decides how the subject is marked.
- **Case** marks a noun’s role by its form (nominative, accusative, dative, genitive).
- `grammar --set feature=value` records an answer, validated against the catalogue.

CHAPTER 14

14

Idioms and metaphors

A language is more than literal meanings. “It’s raining cats and dogs” has nothing to do with animals; “time is money” treats an abstraction as a substance. These **idioms** and **conceptual metaphors** are part of what makes a language feel lived-in and human. Recording a few gives your conlang texture — and helps Inkhaven’s translator render meaning rather than word-for-word nonsense.

Idioms

An **idiom** is a fixed phrase whose meaning is not the sum of its words. You record one with its literal word-by-word reading **and** its real meaning, so both are preserved:

```
inkhaven language idiom-add Eldar --form "kala men" \
    --literal "cold heart" --meaning "unforgiving" --register
formal
```

Here the Eldar phrase **kala men** literally says "cold heart" but means "unforgiving". The optional `--register` marks it as formal speech.

> TERM **Idiom**
>
> A fixed expression whose meaning cannot be worked out from its individual words — English "kick the bucket" means "to die", not anything about buckets. Recording the literal reading and the real meaning separately lets a translator (human or AI) avoid translating it word for word into nonsense.

Conceptual metaphors

A **conceptual metaphor** is a deeper pattern: a whole way of thinking that maps one idea onto another, surfacing in many expressions. English speakers routinely talk about **time** as if it were **money** — you **spend** time, **save** time, find it **wasted**. Declaring such a mapping tells the translator how your speakers think:

```
inkhaven language metaphor-add Eldar --source JOURNEY --target
LIFE \
    --example "she reached a crossroads"
```

This records that your speakers understand **life** (the target) in terms of a **journey** (the source), with an example expression.

TERM **Conceptual metaphor**

A systematic way one domain of experience is understood in terms of another, underlying many everyday expressions — LIFE IS A JOURNEY, ARGUMENT IS WAR ("he **attacked** my point", "I **defended** my claim"). It is written with the **source** (the concrete domain, JOURNEY) mapped onto the **target** (the abstract one, LIFE). Choosing your language's metaphors shapes how its speakers see the world.

Reviewing what you have

List the idioms and metaphors you have recorded with:

```
inkhaven language idioms Eldar
```

Both are stored in the **Grammar** chapter alongside your typology answers. When you translate a paragraph into the language, Inkhaven consults them so the result is idiomatic — reaching for **kala men** where the meaning is "unforgiving", rather than translating "unforgiving" literally.

A LITTLE GOES A LONG WAY

You do not need many. A handful of idioms and one or two governing metaphors give a language a recognisable way of speaking — its own turns of phrase, its own way of carving up experience. They are also a delight to invent: each one is a tiny window into how your speakers think.

WHAT YOU LEARNED

- An **idiom** is a fixed phrase whose meaning is not its literal words; record both with `idiom-add`.
- A **conceptual metaphor** maps a concrete **source** domain onto an abstract **target** (LIFE IS A JOURNEY); record it with `metaphor-add`.
- `idioms` lists what you have; both are stored in the Grammar chapter.
- The translator uses them to render meaning idiomatically rather than literally.

PART V

A History for Your Language

CHAPTER 15

15

Sound change and proto-languages

Real languages have a past. Latin slowly became French, Spanish, and Italian; the single sound /k/ in Latin **centum** became the /s/ of French **cent** and stayed a /k/ in others. Languages change, and the most regular kind of change is in their sounds. Giving your language a history — descending it from an older **proto-language** by a chain of sound changes — is what turns a flat invention into something with depth. This part is optional, but it is where conlanging becomes most rewarding.

TERM Proto-language

An older, ancestral language that later languages descend from. Latin is the proto-language of the **Romance** family (French, Spanish, Italian, …). Proto-languages are sometimes real and recorded (like Latin) and sometimes **reconstructed** — worked out backward from their descendants, like Proto-Indo-European, which no text records.

TERM Sound change

A regular shift in pronunciation that spreads through a language over time, affecting every word that meets its conditions — for example, every /p/ at the end of a word becoming /f/. Because sound changes are **regular**, they are predictable, and they are the engine that turns one language into several.

Sound change is just allophony over time

Here is a piece of good news: a historical sound change is written **exactly** like an allophony rule (Chapter 7). The same `target > result / left _ right` notation applies. The only difference is in the telling — allophony is variation happening **now**, within one language; a sound change is a shift that happened **over generations**, turning a parent language into a child.

Declaring a descent

You give a language a parent by adding a `diachronics` block to its **Phonology** chapter. It names the `proto` (the parent language) and an ordered list of the sound-change `rules` that turned the parent into this language:

```
{ diachronics: {
    proto: "ProtoEldarin"
    rules: [
```

```
      { rule: "p > f / _ #" }
      { rule: "k > h / V _ V" }
    ]
} }
```

This says Eldar descends from a language called **ProtoEldarin**, and two changes happened on the way: every /p/ at the end of a word became /f/, and every /k/ between two vowels became /h/. (You would, of course, first create **ProtoEldarin** as its own language with `language init`, and give it a vocabulary.)

> **THE RULES ARE DEFINED ON THE PARENT'S SOUNDS**
>
> The changes describe what happened to the **proto-language's** sounds, so Inkhaven uses the proto's inventory to read the words before applying the chain. Build the proto first — its phonology and at least some words — then describe how the child diverged from it.

Evolving a word

Watch a single proto-word travel down the chain to its modern form:

```
inkhaven language sound-change Eldar --form tap
```

Inkhaven applies the rules in order and prints the result — `tap > taf` (the final /p/ became /f/). Try it with several words to feel how the chain reshapes the language.

Evolving the whole vocabulary

The real payoff: take the proto-language's **entire** dictionary and evolve it, producing the daughter's vocabulary in one step — each word transformed by the sound changes, with its meaning carried forward and its origin recorded:

```
inkhaven language derive-lexicon Eldar --yes
```

Without `--yes` it only proposes; with it, the words are added to Eldar's dictionary, each remembering the proto-word it came from. In moments you have a vocabulary that is **systematically related** to its ancestor — the hallmark of a real language family, which the next chapter explores.

WHAT YOU LEARNED

- A **proto-language** is an ancestor; **sound changes** are the regular shifts that turn it into descendants.
- A sound change uses the same SPE notation as allophony — change over generations rather than variation now.
- Declare descent with a `diachronics` block: a `proto` and an ordered list of `rules`.
- `sound-change --form` evolves one word; `derive-lexicon` evolves the proto's whole dictionary into the daughter's.

CHAPTER 16

16

Language families

Give one proto-language **two** sets of sound changes and you have two sister languages — a **family**. French and Spanish are sisters, both children of Latin, recognisably related yet distinct. This chapter shows how to grow and explore a family: how related words line up across the daughters, how to draw the family tree, and how Inkhaven can even reconstruct a lost ancestor or judge whether your history is believable.

TERM Language family

A group of languages descended from a common proto-language. Members are **related**; a word that survives from the ancestor into several daughters appears in each as a **cognate**. Examples: the Romance family (from Latin), the Germanic family (English, German, Dutch, …).

TERM Daughter language

A language that descends from a given proto-language. French and Spanish are daughters of Latin. You make a daughter by creating a language whose `diachronics` block names the proto and gives the sound changes that produced it (Chapter 15).

Cognates: the same word, evolved

When a proto-word survives into several daughters, its descendants are **cognates** — the "same word", reshaped differently by each daughter's sound changes. Latin **centum** gives French **cent**, Spanish **ciento**, Italian **cento**: cognates all. Inkhaven traces a proto-form's reflex in every daughter at once:

```
inkhaven language cognates ProtoEldarin --form takap
```

For the proto-form **takap** it applies each daughter's chain and prints the result in each — perhaps `Eldar takaf` versus `Sindarin tahaf`. Lining up cognates this way is exactly how historical linguists study real families, and it is deeply satisfying to watch your own languages diverge from a shared root.

TERM Cognate

A word in one language that descends from the same ancestral word as a word in a related language. English **father**, German **Vater**, and Latin **pater** are cognates — all from one Proto-Indo-European word. Cognates are how relatedness is proven, and how the shape of the proto is recovered.

The family tree

To see the whole family laid out — every language under its declared ancestor — draw the tree:

```
inkhaven language family-tree
```

It prints a genealogical diagram of your languages, each nested under its `proto`. As you add daughters and granddaughters, the tree grows to show the full shape of your invented family.

Looking backward: reconstruction

Historical linguists often work in the other direction: given several cognates, they **reconstruct** the ancestral form that must have produced them. Inkhaven can do this with AI — propose the proto-form from a set of daughter forms:

```
inkhaven language reconstruct --forms "tava taba" --gloss water
```

Given the cognate forms **tava** and **taba** (both meaning "water"), it proposes the most plausible ancestor they could both come from, and explains the reasoning. This is AI-assisted and advisory — a creative aid for designing a deep history.

> TERM **Reconstruction**
>
> Working out the form of an unrecorded ancestral word from its surviving descendants, by reversing the sound changes that produced them. The starred forms in linguistics books (***pater**) are reconstructions. Inkhaven's `reconstruct` proposes one for you from cognate forms.

Is your history believable?

Finally, you can ask the AI whether the chain of sound changes you invented is **typologically plausible** — whether each change is a natural kind that real languages actually undergo, and whether the order makes sense:

```
inkhaven language realism-check Eldar
```

It assesses your sound-change chain and flags anything unnatural, so you can make a history that feels real. Like reconstruction, it is advisory: a knowledgeable second opinion, not a verdict.

> **WHERE THE SUITE SHINES**
>
> Diachronics is where Inkhaven's design pays off most. Because sound change reuses the allophony engine, because daughters reuse the lexicon, and because cognates reuse the change-chains, a few rules give you a whole evolving family for almost no extra effort. A proto plus two daughters is a weekend's work and an enormous boost to realism.

> **WHAT YOU LEARNED**
>
> - A **language family** is a set of daughters from one proto; related words are **cognates**.
> - `cognates <proto> --form` shows a proto-word's reflex in every daughter; `family-tree` draws the whole family.
> - `reconstruct --forms` (AI) proposes a lost ancestor from cognates; `realism-check` (AI) judges whether your sound history is plausible.
> - Two daughters of one proto is a small effort for a large gain in depth.

PART VI

A Writing System

CHAPTER 17

17

Designing glyphs

So far your language has been written in ordinary letters. Now you can give it its own **script** — an alphabet of shapes you design, compiled into a real, usable **font** that renders your words in their native form. This is one of Inkhaven's most striking features, and it does it entirely on its own, with no other software. This part is optional and the most advanced; the language is complete without it.

TERM Script and glyph

A **script** is a system of written symbols for a language (the Latin script, the Cyrillic script, the Korean Hangul script). A **glyph** is one such symbol — one drawn shape. Designing a script means designing its glyphs, usually one per sound or per syllable.

TERM Font

A file containing the drawn glyphs of a script in a form computers can display and print. When you "install a font" you are giving your system a set of glyph shapes tied to characters. Inkhaven compiles your glyph drawings into a standard **TrueType** font that any program can use.

What a glyph file looks like

Each glyph is a small **SVG** file — a vector drawing, the kind made of shapes and curves rather than pixels, so it scales to any size cleanly. A glyph for the sound /a/ might be a simple drawing saved as `a.svg`. You can draw glyphs in any vector editor (Inkscape, Illustrator, and many free tools), or — as we will see — have the AI draft them from a description.

TERM SVG (vector drawing)

Scalable Vector Graphics — a way of describing a drawing as shapes and curves (a filled outline) rather than a grid of pixels. Because it is defined mathematically, it stays crisp at any size, which is exactly what a font needs. Each glyph is one SVG file.

What makes a good font glyph

Not every drawing works as a font glyph. A font glyph must be a **filled shape** — a solid black outline — not a thin line, not a photo, not a coloured gradient. A font is monochrome: it has one ink colour, so a glyph is defined purely by its outline. Before you commit a glyph, check it with the **lint** command:

```
inkhaven language glyph-lint --svg ./a.svg
```

This reports whether the SVG is suitable: it requires filled paths, and flags common mistakes — a stroke-only line (which has no fillable shape), an embedded photo, a gradient, or a near-white fill where you probably meant to cut a hole.

THE WHITE-FILL TRAP

A frequent beginner mistake is to draw the hole in a letter like “O” by painting a white circle over a black one. In a monochrome font this does **not** make a hole — both shapes become solid ink. The correct way to cut a hole (a **counter**) is to draw the inner outline wound in the opposite direction to the outer one. The lint command warns you when it sees a likely white-fill mistake.

TERM **Counter**

An enclosed empty space inside a glyph — the hole in “O”, “D”, or “e”. In a font, a counter is made by drawing the inner outline in the **reverse** direction to the outer outline, not by painting it white. The opposing directions tell the font where the ink is and where the hole is.

Drawing a glyph with AI

If you would rather describe a glyph than draw it, the AI can draft one for you. You give a short description; it produces an SVG, runs it through the same lint check, and shows you the result:

```
inkhaven language glyph-draft Eldar --describe "a vertical stroke with a hook" \
    --phoneme p --out p.svg
```

By default it only drafts and previews. The drawing is advisory — review it, re-run with a better description if you like, and only when you are happy do you keep it. (Adding `--yes`, and only if the draft passes the lint check, binds it straight into your font, which the next chapter explains.) The AI is told to draw filled, monochrome shapes and to cut counters the correct way, so its output is usually ready to use.

> **HAND-DRAWN AND AI GLYPHS MIX FREELY**
>
> You can draw some glyphs yourself and have the AI draft others; both go through the same lint check and into the same font. A common workflow is to let the AI rough out a whole alphabet from descriptions, lint them all, and then redraw by hand the few you want to perfect.

> **WHAT YOU LEARNED**
>
> - A **script** is made of **glyphs**; each is a filled **SVG** vector drawing, one per sound.
> - A font glyph must be a **filled, monochrome shape**; `glyph-lint` checks suitability.
> - Cut holes (**counters**) with reverse-wound outlines, never white fill — lint warns you.
> - `glyph-draft --describe …` has the AI draft a glyph from words; advisory, lint-checked, kept only on your approval.

CHAPTER 18

18

Building the font

You have glyphs. Now you **bind** each one to a sound and a character slot, and compile them into a finished font file. Inkhaven does the whole compilation itself — no font-editing software required — and the result is an ordinary TrueType font you can install, embed in documents, and share.

Binding a glyph

To make a glyph part of your language's writing system, you **import** it: Inkhaven preflights it (the lint check from last chapter), copies it into the language's glyph store, and records which sound it represents and which character slot it occupies.

```
inkhaven language font-import-glyph Eldar --svg ./a.svg --phoneme
a --codepoint a
```

This binds the drawing `a.svg` to the phoneme /a/ and to the character "a". For a sound that has an ordinary letter, you can use that letter as the codepoint, as here. For invented symbols with no letter, you use a number from a special range (explained next).

> TERM **Codepoint**
>
> The unique number a computer uses to identify a character. Every character — "A", "ß", "あ", an emoji — has a codepoint in the **Unicode** standard, written like **U+0041**. A font maps codepoints to glyphs: when text contains a codepoint, the font supplies the matching drawn shape.

> TERM **The Private Use Area**
>
> A block of Unicode codepoints (starting at **U+E000**) deliberately left unassigned, reserved for anyone to use for their own characters. This is exactly where your invented symbols belong — they are real, usable characters, but they do not collide with any standard ones. Bind a glyph with no natural letter to a Private Use codepoint like `U+E000`.

For an invented symbol, bind it to a Private Use codepoint:

```
inkhaven language font-import-glyph Eldar --svg ./sun.svg --
phoneme o --codepoint U+E000 --name sun
```

The font lives in your language

These bindings are stored as a `font` block in the language — a record of the family name, the design grid size, and every glyph with its codepoint and phoneme.

You do not have to write this by hand; `font-import-glyph` builds it for you. To review what you have bound, with the status of each glyph's artwork:

```
inkhaven language font-config Eldar
```

This lists every glyph, its codepoint, the phoneme it stands for, and whether its drawing is present and usable.

Compiling the font

When your glyphs are bound, compile them into a font file straight from the language:

```
inkhaven language font-build --language Eldar --format ttf --out
Eldar
```

This produces `Eldar.ttf`, a standard TrueType font. Inkhaven does the entire compilation in-process: it converts each glyph's outline into font curves, lays them out on the design grid, and assembles a complete font file — with no external program. You can also ask for `ufo` (an editable font **source** you can open in professional font editors) or `both`.

> TERM **TrueType and UFO**
>
> **TrueType** (a `.ttf` file) is the everyday font format your computer already uses — ready to install and use immediately. **UFO** (Unified Font Object) is an editable font **source**, the working format of professional font editors like FontForge or Glyphs. `font-build` can produce either; use TTF to **use** the font, UFO to **keep editing** it.

USING YOUR FONT

The `.ttf` file is a real font. Install it like any other to type your script in any program, or — as Part VII shows — let Inkhaven embed it directly in the PDF books it produces, so your dictionary shows each headword in its own native script. To preview the font in a document, compile a Typst file that uses it with `typst compile --font-path <folder> document.typ`.

WHAT YOU LEARNED

- Bind each glyph to a **phoneme** and a **codepoint** with `font-import-glyph`.
- Ordinary sounds can use their letter; invented symbols use the **Private Use Area** (`U+E000` and up).
- Bindings live in a `font` block; `font-config` lists them with artwork status.
- `font-build --format ttf` compiles a real **TrueType** font in-process; `ufo` gives an editable source.

CHAPTER 19

19

Complex scripts and typing

Not every writing system is a simple row of letters. Korean stacks letters into square syllable blocks; Egyptian hieroglyphs pack signs into rectangular groups. And once you have a font, you will want to **type** in it — to turn romanized text into your script automatically. This chapter covers both: composing complex units, and typing the script.

Composed blocks

Some scripts build one written unit out of several component glyphs arranged in two dimensions — a Korean syllable square is a **lead** consonant, a **vowel**, and a **tail** consonant placed in a grid. Inkhaven supports this with **spatial templates**: named layouts that say where each component goes.

> TERM **Spatial template**
>
> A named layout that divides a square into cells — **left/right**, **top/bottom**, a **2×2 quadrat**, **three stacked rows** — into which component glyphs are placed to build a composite character. Used for syllable-block scripts (like Hangul) and packed scripts (like Egyptian quadrats).

List the available templates, then compose a block by dropping a glyph into each cell:

```
inkhaven language font-templates Eldar
inkhaven language font-compose Eldar --template lr \
    --name ka --codepoint U+AC00 --phoneme ka \
    --slot left=lead --slot right=vowel --yes
```

The built-in templates are `lr` (left/right), `tb` (top/bottom), `quad` (a 2×2 grid), and `stack3` (three rows). This **bakes** the two components into a single new glyph — a precomposed block — that lives in the font alongside its parts.

Layout-time arrangement

Baking every possible block into the font works for a script with a fixed set of syllables, but not for one where signs combine freely and endlessly — like hieroglyphs. For those, Inkhaven can arrange the components at **layout time** instead, emitting a small Typst snippet that places each component in its cell when the document is typeset:

```
inkhaven language spatial-typst Eldar --template tb \
    --name quadrat_sunbar --slot top=sun --slot bottom=bar
```

You drop the snippet into a Typst document, and the quadrat renders with the glyphs stacked — no precomposed glyph required. This is the path for scripts where the number of possible combinations is too large to bake.

Typing the script

Finally, you want to write **in** your script without hunting for codepoints. The **transliterate** command turns romanized text into your script's characters, using the phoneme bindings you made when building the font:

```
inkhaven language transliterate Eldar --text "katha"
```

It reads the romanized input left to right, matching the **longest** glyph key at each step — so a two-letter key like `th` or `ka` wins over the single letters `t` and `h` — and outputs the corresponding string of your script's codepoints. Anything it cannot match passes through unchanged and is flagged, so you know which sounds still need a glyph.

TERM **Transliteration (input method)**

Converting text from one script into another — here, from the Latin letters you type into your invented script's characters. The rule "longest key wins" means a digraph (two letters standing for one sound, like `th`) is matched as a unit before its individual letters. This is the engine behind typing a constructed script.

THIS IS ENOUGH FOR A REAL SCRIPT

With glyphs, a compiled font, optional composed blocks, and transliteration, your language has a complete, usable writing system. The dictionary and grammar books of the next part will show each word in this native script, rendered with the very font you built.

WHAT YOU LEARNED

- **Spatial templates** arrange component glyphs into composite units (Hangul squares, quadrats).
- `font-compose` **bakes** a composed block into the font; `spatial-typst` arranges components at **layout time** for open-ended scripts.
- `transliterate` types the script: romanized text → script codepoints, longest key first.
- Together these give a complete, typeable writing system rendered by your own font.

PART VII

Producing the Books

CHAPTER 20

20

Producing the books

Everything you have built — sounds, words, grammar, history, a script — can now be turned into finished, printable books. Inkhaven produces four kinds of document from your language's own data: a **dictionary**, a **reference grammar**, a **study guide**, and a learner's **tutorial**. Each comes in two flavours: plain Markdown, or a beautifully typeset book (the same B5 design as the book in your hands), ready to compile into a PDF that embeds your conscript font.

A quick word about output formats

Every output command takes `--format md` or `--format typ`. **Markdown** (`md`) is plain text you can read anywhere or paste into other tools. **Typst** (`typ`) produces a typeset book; you turn it into a PDF with the free Typst program:

```
typst compile --font-path <folder-with-your-ttf> book.typ
book.pdf
```

The `--font-path` points Typst at the folder holding your `Eldar.ttf`, so the PDF can render your script. When a command takes `--font Eldar`, it embeds that font and shows your words in their native form.

The dictionary

```
inkhaven language dictionary Eldar --format typ --font Eldar --
out dict.typ
```

This renders your lexicon as a real dictionary: a title page, a table of contents, an overview of the language, and a two-column, alphabetised list of every word with its pronunciation, part of speech, and meaning. When you pass `--font`, each headword also appears in your native script beside its romanized form. Compile it with Typst and you have a printable dictionary of your language.

The reference grammar

```
inkhaven language grammar-book Eldar --format typ --font Eldar --
out grammar.typ
```

The companion volume: a reference grammar drawn entirely from your language's data — the phonology (inventory, syllable structure, phonotactics, allophony, stress, tone), the morphology (affixes and derivations), the typology answers, the idioms and metaphors, and the sample texts. It is a faithful, deterministic description of exactly what you built.

The study guide (AI)

A bare reference grammar can be daunting to a newcomer who does not know what “allophony” or “nominative–accusative alignment” mean. Add `--study` and Inkhaven prepends an AI-written **study guide** that defines and explains every linguistic term the grammar uses, grounded in your language’s own examples:

```
inkhaven language grammar-book Eldar --format typ --font Eldar \
    --study --provider deepseek --out grammar.typ
```

The reference itself stays exactly as before — only the study guide is AI-written, and it needs an AI provider. It turns a technical reference into something a beginner can actually learn from.

The learner's tutorial (AI)

Finally, the gentlest on-ramp of all — a complete beginner’s **textbook** for your language, written by the AI from your data:

```
inkhaven language tutorial Eldar --format typ --font Eldar \
    --provider deepseek --out learn.typ
```

The model authors a graded course: a warm introduction, a pronunciation guide, lessons that **explain** the grammar with worked examples drawn from your words, a reading passage, and a practice exercise per lesson. It is constrained to your language’s actual sounds, words, and rules — it never invents vocabulary or grammar. This is the book you would hand someone who wants to **learn** your language, rather than look things up in it.

DETERMINISTIC WHERE IT COUNTS, AI WHERE IT HELPS

Notice the division of labour. The dictionary and the grammar reference are generated **deterministically** — they are an exact, trustworthy description of your data. The study guide and the tutorial, where the value is in **teaching prose**, are AI-written. You always get an accurate reference; you optionally get a friendly teacher on top. Both kinds of book embed your font and look the same on the shelf.

WHAT YOU LEARNED

- Four documents come from your language: **dictionary**, **grammar reference**, **study guide**, and **tutorial**.
- `--format md` is plain text; `--format typ` is a typeset book you compile with `typst compile --font-path … `.
- `--font Eldar` embeds your script; dictionary and grammar are **deterministic**.
- `grammar-book --study` and `tutorial` are **AI-written** teaching materials (need a provider).

PART VIII

A Complete Walkthrough

CHAPTER 21

21

A complete walkthrough

This chapter builds a whole small language — **Avesha** — from nothing to finished books, in one unbroken sequence. Follow along command by command and you will practise every pillar of the book at once. Everything here uses what the earlier chapters explained; refer back whenever a step needs more detail.

THERE IS A SCRIPT FOR THIS

Inkhaven ships a runnable script that performs this entire walkthrough automatically — `examples/conlang/build-sample-language.sh`, with a checked-in sample of its output in `examples/conlang/sample-output/`. Read this chapter to understand each step; run the script to watch it happen.

1. Project and language

```
inkhaven init ~/avesha-project
cd ~/avesha-project
inkhaven language init Avesha
```

2. Phonology

Create a file in Avesha's `04-phonology` folder with the sound system — eleven sounds, syllable shapes, a couple of constraints, two allophony rules, and penultimate stress:

```
{
  phonemes: [
    { ipa: "p", kind: "consonant" } { ipa: "t", kind:
"consonant" }
    { ipa: "k", kind: "consonant" } { ipa: "s", kind:
"consonant" }
    { ipa: "m", kind: "consonant" } { ipa: "n", kind:
"consonant" }
    { ipa: "l", kind: "consonant" } { ipa: "r", kind:
"consonant" }
    { ipa: "a", kind: "vowel" } { ipa: "i", kind: "vowel" }
{ ipa: "u", kind: "vowel" }
  ]
  classes: { C: ["p","t","k","s","m","n","l","r"], V:
["a","i","u"] }
```

```
  templates: { root: [ { pattern: "C V" }, { pattern: "C V
C" } ] }
  constraints: [ { kind: "no_geminate" }, { kind:
"max_cluster_size", value: 2 } ]
  allophony: [ { rule: "t > s / _ i" }, { rule: "n > m / _ p" } ]
  stress: "penultimate"
}
```

Then adopt it and check:

```
inkhaven reindex --adopt
inkhaven language stats Avesha
```

3. A vocabulary

```
inkhaven language add-word Avesha pata --type noun --translation
stone
inkhaven language add-word Avesha kira --type noun --translation
bird
inkhaven language add-word Avesha suna --type noun --translation
sun
inkhaven language add-word Avesha nami --type verb --translation
see
inkhaven language add-word Avesha palu --type verb --translation
run
inkhaven language add-word Avesha mira --type adjective --
translation bright
inkhaven language audit Avesha
```

4. Grammar

Add a morphology block to the `03-grammar` folder — a dative and a plural suffix, and an agent-noun derivation:

```
{
  morphemes: [
    { id: "dat", gloss: "DAT", form: "ti", position: "suffix" }
    { id: "pl",  gloss: "PL",  form: "u",  position: "suffix" }
  ]
  derivations: [
    { name: "agent", form: "ar", position: "suffix",
      from_pos: "verb", to_pos: "noun", gloss_template: "one who
{}s" }
  ]
}
```

Record the typological choices and reindex:

```
inkhaven reindex --adopt
inkhaven language grammar Avesha --set word_order=sov
inkhaven language grammar Avesha --set
alignment=nominative_accusative
inkhaven language grammar Avesha --set case=yes
```

Watch the allophony fire at an affix boundary — **pata** plus the dative **-ti**:

```
inkhaven language paradigm Avesha --root pata --gloss stone
```

The dative form comes out **patasi**: the /t/ of the root softened to /s/ before the /i/ of the suffix, by the rule you wrote in step 2.

5. A writing system

Draw or AI-draft one glyph per phoneme, bind each to its sound and a Private Use codepoint, then compile the font. With AI drafting:

```
inkhaven language glyph-draft Avesha --describe "a bold vertical
post" \
    --phoneme p --codepoint U+E000 --name p --provider deepseek
--yes
# … one per phoneme …
inkhaven language font-config Avesha
```

```
inkhaven language font-build --language Avesha --format ttf --out
Avesha
```

Type a word in the new script:

```
inkhaven language transliterate Avesha --text kira
```

6. The books

```
inkhaven language dictionary   Avesha --format typ --font Avesha
--out dict.typ
inkhaven language grammar-book Avesha --format typ --font Avesha
--study \
    --provider deepseek --out grammar.typ
inkhaven language tutorial     Avesha --format typ --font Avesha
\
    --provider deepseek --out learn.typ
```

Compile any of them to a PDF that embeds your font:

```
typst compile --font-path . dict.typ dict.pdf
```

You have built a language

Look back over what you did. You chose a sound system and tuned it by ear. You coined words and kept them consistent. You built grammar that inflects words and coins new ones, with sound changes firing automatically at the seams. You set the language's typological character. You designed an alphabet and compiled it into a real font. And you produced a dictionary, a grammar, and a textbook — printable books in your language's own script.

That is a complete constructed language, made from nothing. From here you can go as deep as you like: hundreds more words, a proto-language and a whole family (Part V), idioms and metaphors, a richer script. The tools are the same ones you have now used; only the scale changes.

WHAT YOU LEARNED

- The whole journey is six steps: project, phonology, vocabulary, grammar, writing system, books.
- Each step uses commands from earlier chapters; the allophony, dedup gate, and font compiler all work together automatically.
- The shipped `build-sample-language.sh` runs this end to end.
- You now have everything you need to build your own language from zero.

APPENDIX A

A

Command reference

Every conlang command, grouped by what it does. All take the form `inkhaven language <action> <language> [options]`. Add `--help` to any for full details.

Setup

`init <name>` Create a language sub-book with its five chapters.

`list` List every defined language with summary counts.

Phonology

`generate-word <lang> --role root --count N` Invent word-shapes that obey your templates and constraints.

`syllabify <lang> --word W` Show how a word divides into syllables.

`ipa <lang> --word W` Show a word's **surface** pronunciation after allophony.

`stress <lang> --word W` Mark which syllable carries stress.

`romanize <lang> --text T [--reverse] [--scheme S]` Convert between spelling and sounds.

`tone <lang> --tones "3 3 3"` Apply tone-sandhi rules to a tone sequence.

Lexicon

`add-word <lang> <word> --type POS --translation M` Add a dictionary entry (or `--import file.csv`).

`remove-word <lang> <word>` Delete an entry.

`query <lang> [--pos] [--register] [--domain] [--era] [--text] [--json]`
Search the lexicon.

`generate-lexicon <lang> --topic T --count N [--semantic] [--yes]` AI-generate themed words behind the dedup gate.

`audit <lang> [--json]` Report phonotactic violations, homophones, duplicate meanings.

`scan-manuscript <lang> [--json]` Find language-like words in your prose that are not yet defined.

`stats <lang> [--json]` A descriptive profile of the language's sounds and words.

Grammar

`paradigm <lang> --root R --template T --gloss G` Generate a word's inflected forms.

`agree <lang> --word W --pos P --features "number=pl,case=nom" [--gloss G]` Inflect a dependent word to agree with its head's features.

`gloss <lang> --text "…"` Interlinear (word-by-word) gloss of a sentence.

`derive <lang> --root R --gloss G --pos P [--yes]` Coin derived words from a root.

`grammar <lang> [--set feature=value]` View or set the typology questionnaire.

`idiom-add <lang> --form F --literal L --meaning M [--register R]` Record an idiom.

`metaphor-add <lang> --source S --target T [--example E]` Record a conceptual metaphor.

`idioms <lang>` List recorded idioms and metaphors.

Diachronics

`sound-change <lang> --form W` Evolve one proto-form through the language's sound-change chain.

`derive-lexicon <lang> [--yes]` Evolve the proto's whole dictionary into this daughter.

`family-tree` Draw the genealogical tree of all languages.

`cognates <proto> --form W` Trace a proto-form's reflex in every daughter.

`reconstruct --forms "…" [--gloss G]` (AI) Propose a proto-form from cognates.

`realism-check <lang>` (AI) Assess whether the sound-change chain is plausible.

Writing systems

`glyph-lint --svg F` Check whether an SVG is suitable as a font glyph.

`glyph-draft <lang> --describe "…" --phoneme P [--codepoint C] [--out F] [--yes]`
(AI) Draft a glyph from a description.

`font-import-glyph <lang> --svg F --phoneme P [--codepoint C] [--name N]`
Bind a glyph into the font.

`font-config <lang> [--json]` List the font's glyph bindings and artwork status.

`font-build --language <lang> --format ufo|ttf|both [--out F] [--upm N]`
Compile the font.

`font-templates <lang>` List spatial templates for composed blocks.

`font-compose <lang> --template T --name N --slot SLOT=GLYPH … [--yes]`
Bake a composed block glyph.

`spatial-typst <lang> --template T --name N --slot SLOT=GLYPH …`
Emit a layout-time Typst quadrat.

`transliterate <lang> --text "…" [--json]` Type romanized text into the script's codepoints.

Output

`dictionary <lang> --format md|typ [--out F] [--font Fam]` Render the dictionary.

`grammar-book <lang> --format md|typ [--out F] [--font Fam] [--study --provider P]` Render the reference grammar (with optional AI study guide).

`tutorial <lang> --format md|typ [--out F] [--font Fam] [--provider P]` (AI) Render a learner's textbook.

Worldbuilding links

`link-place <place> <lang> [--secondary]` Link a place to a language spoken there.

`link-character <char> <lang> <level>` Record a character’s fluency in a language.

`speakers <lang>` List who speaks a language.

APPENDIX B

B

Configuration block reference

The HJSON blocks you place into your language's chapters, in one place. Remember the rule: quote short word-like values (`kind: "consonant"`), and run `inkhaven reindex --adopt` after editing files by hand.

Phonology chapter

The full phonology block. Every field except `phonemes` is optional.

```
{
  phonemes: [
    { ipa: "k", romanize: "k", kind: "consonant" }   // kind:
consonant | vowel
    { ipa: "a", romanize: "a", kind: "vowel" }
  ]
  classes: { C: ["k"], V: ["a"] }                     // named
phoneme groups
  templates: { root: [ { pattern: "C V (C)", weight: 1.0 } ] }
  constraints: [
    { kind: "max_cluster_size", value: 2 }
    { kind: "no_geminate" }
    { kind: "forbid_in_coda", classes: ["Stop"] }     // or
forbid_in_onset
    { kind: "sonority_sequencing" }
  ]
  allophony: [ { name: "palatalization", rule: "k > tʃ / _ i" } ]
  stress: "penultimate"        // initial | final | penultimate |
antepenultimate | latin
  romanizations: [
    { name: "default"
      mappings: [ { ipa: "k", roman: "c" } ]
      contextual: [ { roman: "c", ipa: "s", before:
"FrontV" } ] }
  ]
  default_romanization: "default"
  tone: { kind: "contour", tones: ["1","2","3"], sandhi:
[ { rule: "3 > 2 / _ 3" } ] }
}
```

A separate block, also in the Phonology chapter, declares descent from a proto:

```
{ diachronics: {
    proto: "ProtoEldarin"
    rules: [ { rule: "p > f / _ #" }, { rule: "k > h / V _ V" } ]
} }
```

The `font` block (built for you by `font-import-glyph`) also lives here:

```
font: {
  family: "Eldar"
  upm: 1000
```

```
  glyphs: [
    { name: "a", codepoint: "a", phoneme: "a" }
    { name: "sun", codepoint: "U+E000", phoneme: "o" }
  ]
}
```

Grammar chapter

The morphology block — affixes, processes, paradigms, derivations, agreement:

```
{
  morphemes: [
    // Concatenative affixes. position: prefix | suffix | infix |
circumfix.
    // Optional precedence: 0 = any (keep declared order), 1 =
next to the root, …
    // Optional category / value tag the affix for the grammar
book.
    { id: "dat", gloss: "DAT", form: "ti", position: "suffix",
      precedence: 1, category: "case",   value: "dative" }
    { id: "pl",  gloss: "PL",  form: "i",  position: "suffix",
      precedence: 2, category: "number", value: "plural" }
    { id: "ag",  gloss: "AG",  form: "um", position: "infix",
anchor: "before_first_vowel" }
    { id: "ptcp", gloss: "PTCP", form: "ge_t", position:
"circumfix" }   // ge_ + stem + _t
    // Non-concatenative processes.
    { id: "pst", gloss: "PST", process: "ablaut", rules:
[ { rule: "i > a" } ] }
    { id: "rdp", gloss: "PL",  process: "reduplication",
reduplicate: "initial_cv" }
    // reduplicate: full | initial_cv | initial_syllable |
final_syllable
  ]
  paradigms: [ { name: "noun", cells: [
    { features: { number: "sg", case: "nom" }, morphemes: [] }
    { features: { number: "pl", case: "dat" }, morphemes:
["dat","pl"] }
  ] } ]
```

```
  derivations: [
    { name: "agent", form: "ron", position: "suffix",
      from_pos: "verb", to_pos: "noun", gloss_template: "one who
{}s" }
  ]
  agreement: [
    { dependent: "adjective", head: "noun", features:
["number","case"], paradigm: "adj" }
  ]
}
```

The typology answers (written for you by `grammar --set`) and idioms / metaphors (written by `idiom-add` / `metaphor-add`) are also stored in this chapter.

Dictionary chapter

Each word is one small block (written for you by `add-word`):

```
{ word: "makil", type: "noun", translation: "sword",
  register: "formal", domain: ["weapon"], era: "third_age" }
```

SPE rule notation

Used by `allophony`, `diachronics`, and tone `sandhi`:

```
TARGET > RESULT / LEFT _ RIGHT
```

`_` the position of the target sound.
`#` a word boundary (start or end of a word).
`∅` or `0` the empty string — insertion (on the left) or deletion (on the right).
a class name (`V`, `C`, `Stop`) matches any sound in that class.
a bare phoneme (`i`, `k`) matches just that sound.

APPENDIX C

C

Glossary of terms

Every linguistic and technical term used in this book, defined in one place, in alphabetical order. Italicised words within a definition have their own entry.

Affix A morpheme attached to a root — a **prefix** (front), **suffix** (end), or infix (middle).

Agent noun A noun naming the doer of an action, derived from a verb (**teach** → **teacher**).

Allophone One of the several ways a single **phoneme** is actually pronounced, chosen by context.

Allophony The system of context-dependent pronunciation variants within a language.

Alignment How a language marks the subject of a sentence — **nominative–accusative** or **ergative–absolutive**.

API key A secret password that lets your copy of Inkhaven use an **AI provider**.

Cognate A word descended from the same ancestral word as a word in a related language (**father** / Latin **pater**).

Codepoint The unique number identifying a character in the Unicode standard (written like **U+0041**).

Conceptual metaphor A systematic understanding of one domain in terms of another (LIFE IS A JOURNEY).

Conlang Short for **constructed language** — a language built on purpose.

Consonant A sound made by obstructing the airflow (p, t, k, s, m, …).

Constraint A single **phonotactic** rule that rejects certain words.

Constructed language A language deliberately created, with invented vocabulary and grammar.

Counter The enclosed empty space inside a glyph (the hole in “O”), cut with a reverse-wound outline.

Daughter language A language descended from a given **proto-language**.

Dedup gate The automatic checks that stop word generation from ever creating a duplicate.

Derivation Forming a genuinely new word from an existing one (**build** → **builder**).

Diachronics The historical change of a language over time, and how it splits into a family.

Domain The subject area a word belongs to (weapon, kinship, weather).

Etymology The origin and history of a word — what it descended or was derived from.

Font A file of drawn glyphs that computers display and print.

Geminate A doubled or lengthened consonant (Italian **gatto**).

Gloss (grammatical) A short small-capitals label for a grammatical piece (PL, DAT, PST).

Gloss (meaning) A short statement of a word’s meaning in your working language.

Glyph One drawn symbol of a **script**.

Grammar The rules for how words change form and combine into sentences.

Grammatical case Marking a noun’s role by changing its form (nominative, accusative, dative, genitive).

Homophone A word that sounds the same as another but means something different.

HJSON A relaxed, human-readable data format used to define a language's blocks.

Idiom A fixed phrase whose meaning is not the sum of its words.

Inflection Changing a word's form to express grammar, without making a new word (**cat** / **cats**).

Interlinear gloss A word-by-word breakdown lined up under a sentence (Leipzig glossing).

IPA The International Phonetic Alphabet — one symbol per speech sound.

Language family A group of languages descended from a common **proto-language**.

Lexicon The full vocabulary of a language; a single word is a **lexeme**.

Linguistics The scientific study of human language.

Morpheme The smallest unit of a word that carries meaning.

Morphology How words are built from morphemes and change shape for grammar.

Naturalism Making a constructed language feel like it could be a real human language.

Nucleus The vowel at the centre of a **syllable**.

Onset The consonant(s) before the vowel in a **syllable**.

Coda The consonant(s) after the vowel in a **syllable**.

Paradigm The organised set of all inflected forms of a word.

Part of speech The grammatical category of a word (noun, verb, adjective, adverb).

Phoneme A single distinct sound that can change a word's meaning.

Phoneme class A named group of phonemes that behave alike for some rule (C, V).

Phonology The sound system of a language.

Phonotactics The rules for which sound sequences a language allows.

Private Use Area A range of Unicode codepoints (from **U+E000**) reserved for invented characters.

Proto-language An ancestral language that later languages descend from.

Reconstruction Working out an unrecorded ancestral word from its surviving descendants.

Register The level of formality a word belongs to (formal, neutral, vulgar, sacred, archaic).

Romanization Writing a language's sounds with the Latin alphabet.

Script A system of written symbols for a language.

Sonority How open and resonant a sound is; clusters tend to rise toward the vowel.

Sound change A regular historical shift in pronunciation that turns one language into another.

SPE notation The shorthand `target > result / left _ right` for sound rules.

Stress The relative emphasis given to one syllable of a word.

Surface form How a word is actually pronounced after allophony rules apply.

SVG Scalable Vector Graphics — a drawing made of shapes and curves; each glyph is one.

Syllable A beat of speech built around a vowel (onset + nucleus + coda).

Syllable template A pattern of class names describing an allowed syllable shape (C V (C)).

Syntax The part of grammar dealing with word order and sentence structure.

Tone The use of pitch to distinguish word meanings.

Transliteration Converting text from one script into another (here, Latin → your script).

TrueType An everyday font format (`.ttf`), ready to install and use.

Typology The classification of languages by their structural features.

UFO Unified Font Object — an editable font source format.

Underlying form The abstract phoneme sequence a word is built from, before allophony.

Vowel A sound made with open, unobstructed airflow (a, e, i, o, u).

Word order The default order of subject, verb, and object (SOV, SVO, …).

Word root The core form of a word, carrying its basic meaning, before affixes.